Software Engineering Lab: Release 1 Submission

AI-Dev-Tracker: AI-Assisted Software Development Traceability Tool

Team 6 (six seven)

March 17, 2026

1 Abstract

This document serves as the formal project proposal for Release 1 of the AI-Dev-Tracker. The project focuses on creating a CLI-based ecosystem that bridges the gap between AI-assisted development and version control systems. It outlines a methodology for recording prompt history, linking interactions to Git commits, and measuring AI dependency to identify developer struggle patterns and quantify AI contribution to software projects.

2 Problem Statement

Modern software development is rapidly evolving with the integration of Artificial Intelligence tools such as ChatGPT, Claude, GitHub Copilot, and Gemini. These tools assist developers by generating code, explaining algorithms, debugging errors, and suggesting improvements. While these AI tools significantly improve productivity, they also introduce several challenges in tracking and managing AI-assisted development.

Currently, there is no standardized mechanism to monitor how AI contributes to software development. Developers frequently interact with AI tools during coding, but these interactions are typically not recorded or linked to the actual code that is eventually committed to version control systems like Git.

This creates several major issues:

1. **Lack of Prompt History Tracking:** Developers often ask multiple prompts to AI systems while solving a problem. However, there is no systematic way to maintain a history of these prompts.

2. **No Link Between AI Interaction and Code Changes:** AI responses that influence code development are not connected to specific commits or files in version control systems.

3. **Difficulty in Identifying AI-Generated Code:** In collaborative projects, it becomes difficult to determine which parts of the code were written by developers and which were generated or influenced by AI tools.

4. **Lack of Insight into Developer Struggles:** Frequent AI queries often indicate areas where developers struggled. Without tracking these interactions, it becomes impossible to analyze these patterns.

5. **Absence of AI Contribution Metrics:** There is currently no reliable way to measure the actual impact of AI assistance on the final software product.

Affected Users / Stakeholders: The following stakeholders are directly affected by this problem:

- Software developers using AI tools
- Development teams working on collaborative repositories
- Researchers studying AI-assisted development
- Software engineering educators analyzing development practices
- Organizations concerned about code traceability and accountability

Impact of the Problem: If this issue remains unresolved, AI usage in development will remain untraceable, making it difficult to audit software development processes and research programming productivity. Teams may face accountability issues regarding AI-generated code, and developer learning patterns and struggles will remain hidden. Therefore, a system that can track, analyze, and measure AI involvement in software development is necessary.

3 Proposed Solution

To address the above problems, our team proposes the development of a tool called AI-Dev-Tracker, a command-line based system designed to monitor and analyze AI-assisted development activity.

High-Level Description: The proposed system will allow developers to log AI prompts and responses through a command-line interface. Each prompt interaction will be stored along with metadata such as file name, timestamp, and Git commit hash. The system will then analyze the similarity between AI responses and the final code to determine whether the AI contributed to that portion of the project.

Core Features:

- **CLI Based Prompt Logging:** Developers can log prompts directly through the command line interface with associated metadata.
- **AI Response Tracking:** Every prompt is stored together with the corresponding AI response to maintain a complete history.
- **Git Commit Integration:** Each interaction is associated with a Git commit hash to enable traceability between AI assistance and version control history.
- **File-wise Prompt Mapping:** Prompts are mapped to specific files within the project for file-level analysis.
- **Base Prompt–Driven Prompt Classification:** The first prompt for a file is treated as the base prompt; subsequent prompts are analyzed relative to it to understand the evolution of AI assistance.
- **File Dependency Detection:** Analyzes relationships between files to detect dependencies and provide architectural insight.
- **Developer Struggle Identification:** Identifies files with high prompt density or repeated attempts as potential areas of struggle.
- **AI Contribution Reporting:** Generates reports summarizing AI involvement, prompt density, and contribution patterns across modules.
- **Offline Mock AI Support:** A mock AI engine allows for consistent and reproducible testing without external API dependencies.

Novelty of the Approach: The proposed system integrates interaction logging with version control, automatically detects AI contribution, and enables behavioral analysis through interaction patterns.

Expected Impact: The system introduces transparency, helps identify struggle areas, provides file-level contribution analysis, and offers enhanced traceability for accountability and research.

4 System Architecture

The system architecture consists of several interconnected components that handle prompt logging, AI response management, Git integration, and similarity analysis.

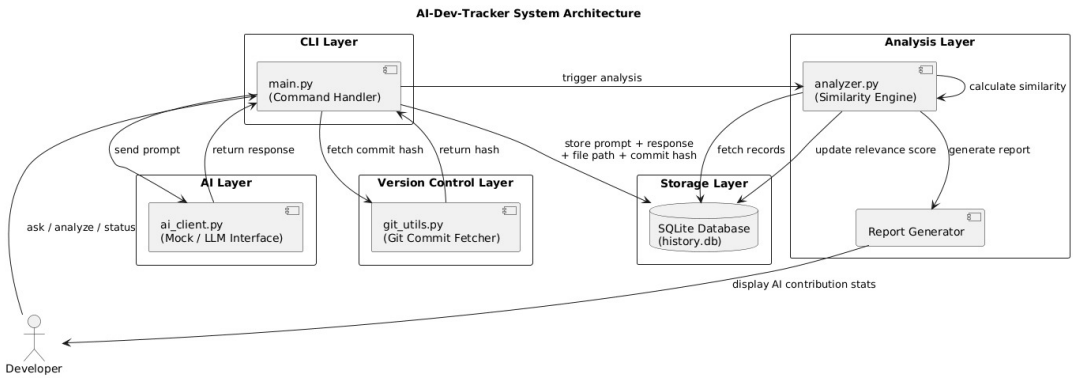

Figure 1: AI-Dev-Tracker System Architecture

Major System Components:

1. **CLI Interface:** The entry point for submissions, triggers logging, and provides commands for analysis and reporting.
2. **Prompt Logger:** Records prompt text, AI responses, and metadata such as timestamps and file names.
3. **Git Integration Module:** Retrieves commit hashes to maintain traceability between prompts and code changes.
4. **AI Engine / Mock AI Module:** Simulates AI responses for stable testing outputs.
5. **Similarity Analyzer:** Compares responses with project files using SequenceMatcher to classify contributions.
6. **Contribution Analyzer:** Aggregates data to measure AI influence percentages and identify usage patterns.
7. **Reporting Module:** Produces developer-readable summaries and highlights AI-dependent modules.

5 Workflow Chart Implementation

The operational flow of the system ensures data integrity through a validated logic path for all developer interactions.

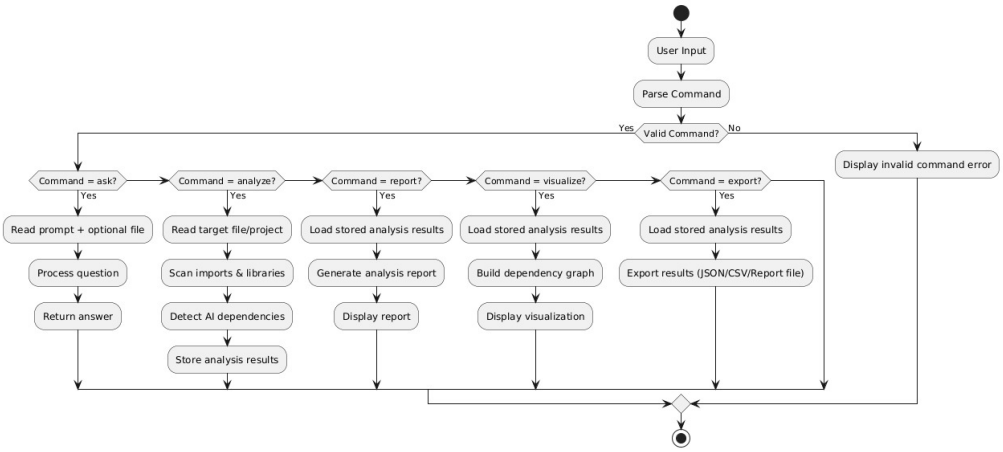

Figure 2: AI-Dev-Tracker Operational Workflow

Data Flow Description:

1. Developer enters a prompt through the CLI.
2. Prompt Logger stores the prompt and associated metadata.
3. The AI Engine generates a response.
4. Git Integration records the corresponding commit hash.
5. Similarity Analyzer compares the response with project files to determine relevance.
6. Contribution Analyzer determines the degree of influence.
7. The Reporting module generates the final usage reports.

6 Tech Stack

- **Backend:** Python (Core logic, string processing, and CLI framework).
- **Database:** SQLite (Local interaction history and project metadata).
- **Version Control:** Git (Integrated via commit hash tracking).
- **AI / ML:** OpenAI, HuggingFace, and custom Mock AI for offline stability.
- **Libraries:** `difflib` for SequenceMatcher-based similarity scoring.

7 Team Roles

Member Name	Role Title	Release 1 Tasks Completed

Shivam	Lead Architect	Designed architecture, implemented modular CLI, and built SQLite logging.
Suhail	Database Lead	Refined schema, tested API integrations, and verified Git functionality.
Sreeja	Documentation + Analyser Logic	Prepared diagrams, technical documentation, and refined relevance metrics.
Anil	QA & Validation	Verified logging accuracy, identified edge cases, and refined error handling.
Harsith	Research Lead	Studied architectural drift and researched similarity detection approaches.

8 AI Usage Declaration

The CLI framework's command-registry pattern was developed with AI assistance for better extensibility and manually refactored. Similarity detection logic was researched using AI tools to determine the hybrid approach (SequenceMatcher and keywords) and manually implemented. All mock AI engine components were written manually based on team discussion.

9 Prompts

Tool	**Prompt Text**	**Purpose**	**Outcome**
ChatGPT	"Suggest a SQL schema for logging AI prompts with file mapping."	Database design.	Used
Gemini	"How to link Python functions to Git commit hashes?"	Git integration.	Used
Claude	"Explain SequenceMatcher ratio for comparing code snippets."	Metric refinement.	Modified

10 Milestones & Timeline

Milestone	Deliverables	Timeline	Lead
Release 1	Core CLI, SQLite Logs, Basic Analysis	Mar 17	Shivam
Intermediate	Session management & Model switching	Mar 24	Suhail
Release 2	Full Reporting, Final Evaluation	TBD	All

11 Goals for Release 2

- **Session Management:** Implement `session_id` and `project_name` for project-level log isolation.
- **Struggle Detection Tuning:** Introduce a 30-minute threshold for sustained struggle detection.
- **Model Selection:** Implement a CLI flag to allow users to toggle between Gemini Flash, Pro, and Mock models.
- **Export Functionality:** Finalize the `export` command to generate formatted CSV and PDF reports.

12 Submission Checklist

Problem Statement — specific and stakeholder-focused.

Proposed Solution — clear and novel approach to traceability.

System Architecture — diagrams and component descriptions included.

Tech Stack — all technologies listed and justified.

Team Roles — all 5 members with distinct responsibilities.

AI Usage Declaration — transparently documented.

Prompts — full log of all key AI interactions included.

Milestones — complete timeline for the project lifecycle.

Release 2 Goals — specific, measurable commitments.